MPI Topology Setup for Sensor Grid

https://gemini.google.com/app/badf39c015a17926

User prompt: I am building a C program using Open MPI to model a sensor grid and a base station process. Write the boilerplate setup code using MPI_Comm_split to separate the base station rank from the grid processes. Then, use MPI_Cart_create to organize the grid nodes into a 2D mesh, and MPI_Cart_shift to identify neighbor ranks in four directions (up, down, left, right).

Response:

```c
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char *argv[]) {
    int world_rank, world_size;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    // Require at least 2 processes: 1 Base Station + 1 or more Grid nodes
    if (world_size < 2) {
        if (world_rank == 0) {
            fprintf(stderr, "Error: Launch at least 2 processes (1 Base Station + Grid nodes).\n");
        }
        MPI_Finalize();
        return EXIT_FAILURE;
    }

    // ------------------------------------------------------------------------
    // 1. MPI_Comm_split: Separate Base Station (rank 0) from Grid nodes (rank >= 1)
    // ------------------------------------------------------------------------
    int color = (world_rank == 0) ? 0 : 1;
    MPI_Comm sub_comm;

    // Split MPI_COMM_WORLD into two sub-communicators based on color
    MPI_Comm_split(MPI_COMM_WORLD, color, world_rank, &sub_comm);

    if (color == 0) {
        // --------------------------------------------------------------------
        // Base Station Process
        // --------------------------------------------------------------------
        printf("[Base Station] Operating on world rank %d\n", world_rank);

    } else {
        // --------------------------------------------------------------------
        // Grid Sensor Processes
        // --------------------------------------------------------------------
        int grid_size;
        MPI_Comm_size(sub_comm, &grid_size);

        // Automatically determine optimal 2D dimensions for grid_size
        int dims[2] = {0, 0};
        MPI_Dims_create(grid_size, 2, dims);

        // --------------------------------------------------------------------
        // 2. MPI_Cart_create: Map grid processes into a 2D mesh topology
        // --------------------------------------------------------------------
        int periods[2] = {0, 0}; // 0 = Non-periodic mesh (boundaries return MPI_PROC_NULL)
        int reorder = 1;         // 1 = Allow MPI to optimize rank assignment to physical CPU topology
        MPI_Comm cart_comm;

        MPI_Cart_create(sub_comm, 2, dims, periods, reorder, &cart_comm);

        // Get Cartesian rank and 2D coordinates for this process
        int cart_rank;
        int coords[2];
        MPI_Comm_rank(cart_comm, &cart_rank);
        MPI_Cart_coords(cart_comm, cart_rank, 2, coords);

        // --------------------------------------------------------------------
        // 3. MPI_Cart_shift: Find neighbor ranks in 4 directions
        // --------------------------------------------------------------------
        // Dimension 0 (Rows): Up (source, displacement -1) / Down (dest, displacement +1)
        int up_rank, down_rank;
        MPI_Cart_shift(cart_comm, 0, 1, &up_rank, &down_rank);

        // Dimension 1 (Columns): Left (source, displacement -1) / Right (dest, displacement +1)
        int left_rank, right_rank;
        MPI_Cart_shift(cart_comm, 1, 1, &left_rank, &right_rank);

        printf("[Grid Node] World Rank: %d | Cart Rank: %d | Coords: (%d, %d) | "
               "Neighbors -> Up: %d, Down: %d, Left: %d, Right: %d\n",
               world_rank, cart_rank, coords[0], coords[1],
               up_rank, down_rank, left_rank, right_rank);
```

```
        MPI_Comm_free(&cart_comm);
    }

    MPI_Comm_free(&sub_comm);
    MPI_Finalize();
    return 0;
}
```

Key Execution Details

1. **`MPI_Comm_split`**: Assigns `color = 0` to Rank 0 (Base Station) and `color = 1` to all remaining processes. This isolates communication between sensor nodes without interfering with the base station.
2. **`MPI_Cart_create`**: Constructs the Cartesian topology on the split grid communicator `sub_comm` using dimensions auto-calculated by `MPI_Dims_create`.
3. **`MPI_Cart_shift`**: Retrieves standard 4-way mesh neighbors. If a process lies on an edge (e.g., top row with no `Up` neighbor), the shift returns `MPI_PROC_NULL` (`-2`), which standard MPI send/receive routines automatically ignore without throwing errors.

Compilation and Execution

```
mpicc -o sensor_grid sensor_grid.c
mpirun -np 5 ./sensor_grid   # 1 Base Station + 4 Grid nodes (2x2 mesh)
```

User prompt: Explain how to structure my MPI communicator creation so that the code can seamlessly switch between a 2D grid topology (e.g., 8×8) and a 1D line topology (e.g., 1×64) without rewriting the core communication logic

Response: To seamlessly switch between a 2D grid and a 1D line topology without changing your core communication logic, **always treat 1D as a degenerate 2D topology ($1 \times N$ or $N \times 1$)** and rely on MPI's native `MPI_PROC_NULL` semantics.

Core Concept: The Degenerate 2D Approach

By fixing the number of Cartesian dimensions (`ndims`) to `2` across your entire application, a 1D line (1×64) is simply a 2D grid where `dims[0] = 1` and `dims[1] = 64`.

When you call `MPI_Cart_shift` on dimension 0 (vertical) of a 1×64 grid:

1. MPI automatically sets both the `Up` and `Down` neighbor ranks to `MPI_PROC_NULL` because there are no rows above or below row 0.
2. MPI point-to-point routines (`MPI_Sendrecv`, `MPI_Isend`, `MPI_Irecv`) natively treat `MPI_PROC_NULL` as a **valid no-op rank**.
3. Any send or receive targeting `MPI_PROC_NULL` returns immediately without transferring data or throwing errors, while operations targeting valid ranks (`Left` and `Right`) execute normally.

This completely eliminates the need for `if/else` checks inside your performance-critical communication loops.

Implementation Architecture

1. Dynamic Topology Setup

Accept grid dimensions as runtime parameters (or calculate them dynamically with `MPI_Dims_create`). Pass these dimensions into `MPI_Cart_create` while keeping `ndims = 2`.

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

// Direction indices for clean array access
typedef enum {
    DIR_UP = 0,
    DIR_DOWN,
    DIR_LEFT,
    DIR_RIGHT
} Direction;

typedef struct {
    MPI_Comm comm;
    int rank;
    int coords[2];
    int neighbors[4]; // Holds UP, DOWN, LEFT, RIGHT ranks
} GridTopology;

GridTopology setup_grid_topology(MPI_Comm sub_comm, int rows, int cols, int periodic_y, int periodic_x) {
    GridTopology grid;

    int dims[2] = {rows, cols};
    int periods[2] = {periodic_y, periodic_x}; // 0 for mesh, 1 for torus/ring
    int reorder = 1;
```

```
    // Always create a 2D Cartesian communicator
    MPI_Cart_create(sub_comm, 2, dims, periods, reorder, &grid.comm);

    MPI_Comm_rank(grid.comm, &grid.rank);
    MPI_Cart_coords(grid.comm, grid.rank, 2, grid.coords);

    // Shift along dimension 0 (Rows: UP / DOWN)
    MPI_Cart_shift(grid.comm, 0, 1, &grid.neighbors[DIR_UP], &grid.neighbors[DIR_DOWN]);

    // Shift along dimension 1 (Cols: LEFT / RIGHT)
    MPI_Cart_shift(grid.comm, 1, 1, &grid.neighbors[DIR_LEFT], &grid.neighbors[DIR_RIGHT]);

    return grid;
}
```

2. Unified Communication Logic (Zero Conditionals)

Because neighbor ranks are stored in a predictable array structure, your halo exchange routine remains 100% identical regardless of whether you run an 8×8 mesh, a 1×64 line, or a 64×1 column.

```
void perform_halo_exchange(GridTopology *grid, double *send_bufs[4], double *recv_bufs[4], int count) {
    MPI_Request reqs[8];
    int req_cnt = 0;

    // Non-blocking halo exchange across all 4 directions
    for (int dir = 0; dir < 4; dir++) {
        int neighbor = grid->neighbors[dir];

        // If neighbor is MPI_PROC_NULL (e.g., UP/DOWN in a 1x64 grid),
        // MPI_Irecv and MPI_Isend instantly return a completed request.
        MPI_Irecv(recv_bufs[dir], count, MPI_DOUBLE, neighbor, 0, grid->comm, &reqs[req_cnt++]);
        MPI_Isend(send_bufs[dir], count, MPI_DOUBLE, neighbor, 0, grid->comm, &reqs[req_cnt++]);
    }

    // Wait for all transfers to complete
    MPI_Waitall(req_cnt, reqs, MPI_STATUSES_IGNORE);
}
```

Switching Topologies at Runtime

You can switch layouts simply by changing command-line arguments or runtime values:

```
int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    // Parse requested shape from user (e.g., 8 8 for 2D, 1 64 for 1D line)
    int rows = (argc > 1) ? atoi(argv[1]) : 1;
    int cols = (argc > 2) ? atoi(argv[2]) : 64;

    // Create the topology
    GridTopology grid = setup_grid_topology(MPI_COMM_WORLD, rows, cols, 0, 0);

    // Core solver execution remains untouched
    // perform_halo_exchange(&grid, send_bufs, recv_bufs, halo_size);

    MPI_Comm_free(&grid.comm);
    MPI_Finalize();
    return 0;
}
```

Summary of Benefits

- **No Branch Mispredictions:** Inner computation and communication loops carry zero `if (dim == 1)` branches.
- **Flexible Periodic Boundaries:** Standard 1D lines, 1D rings, 2D meshes, and 2D toruses are all handled by adjusting `dims` and `periods` arrays.
- **Uniform Memory Allocations:** Buffer arrays for the 4 directions can stay static in size; unused direction buffers simply remain untouched during $1 \times N$ runs.

User prompt: I need to design a hybrid MPI + OpenMP process representing an EV charging station. Thread 0 should be dedicated to handling all MPI messaging, while k worker threads each simulate an individual charging port. Provide an OpenMP C code snippet using #pragma omp parallel that safely shares port status flags in an array without mutex locks, relying on atomic updates

```
Response: #include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <omp.h>
#include <mpi.h>
```

```
#define NUM_PORTS 4 // Number of worker threads (k = 4)
#define BASE_STATION_RANK 0

// Port status flags
typedef enum {
    PORT_IDLE = 0,
    PORT_CHARGING,
    PORT_COMPLETE,
    PORT_FAULT
} PortStatus;

int main(int argc, char *argv[]) {
    int provided;

    // Request MPI_THREAD_FUNNELED since only master thread (Thread 0) makes MPI calls
    MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &provided);
    if (provided < MPI_THREAD_FUNNELED) {
        fprintf(stderr, "Error: MPI implementation does not provide MPI_THREAD_FUNNELED support.\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    int world_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    // Shared status array across all threads
    int port_status[NUM_PORTS] = {PORT_IDLE};

    // Spawn 1 messaging thread (Thread 0) + k worker threads
    #pragma omp parallel num_threads(NUM_PORTS + 1) shared(port_status)
    {
        int tid = omp_get_thread_num();

        if (tid == 0) {
            // ================================================================
            // Thread 0: Dedicated MPI Messaging Handler
            // ================================================================
            int local_snapshot[NUM_PORTS];
            int active = 1;

            while (active) {
                // Safely sample status flags using lock-free atomic reads
                int completed_count = 0;
                for (int i = 0; i < NUM_PORTS; i++) {
                    #pragma omp atomic read
                    local_snapshot[i] = port_status[i];

                    if (local_snapshot[i] == PORT_COMPLETE || local_snapshot[i] == PORT_FAULT) {
                        completed_count++;
                    }
                }

                // Transmit updated status payload to base station
                MPI_Send(local_snapshot, NUM_PORTS, MPI_INT, BASE_STATION_RANK, 0, MPI_COMM_WORLD);

                if (completed_count == NUM_PORTS) {
                    active = 0; // Terminate monitoring when all ports finish
                }

                usleep(100000); // 100ms polling interval
            }
        } else {
            // ================================================================
            // Threads 1..k: Individual Charging Port Simulators
            // ================================================================
            int port_id = tid - 1; // Map thread ID (1..k) to port index (0..k-1)

            // 1. Mark port as CHARGING
            #pragma omp atomic write
            port_status[port_id] = PORT_CHARGING;

            // Simulate charging duration
            usleep(200000 + (port_id * 100000));

            // 2. Mark port as COMPLETE via lock-free atomic update
            #pragma omp atomic write
            port_status[port_id] = PORT_COMPLETE;
        }
    }

    MPI_Finalize();
    return 0;
}
```

Key Technical Aspects

- **`MPI_THREAD_FUNNELED`**: Guaranteed level of thread safety where only the main thread (Thread 0) executes MPI calls, avoiding multi-threaded MPI locking overhead.
- **`#pragma omp atomic write` & `read`**: Guarantees atomic accesses to elements in `port_status[]` without mutex locks (`omp_lock_t`). OpenMP 3.1+ atomic reads and writes implicitly enforce memory visibility across threads for the targeted variable.
- **False Sharing Mitigation**: If high-frequency updates occur, align array memory elements to cache-line boundaries to prevent invalidation contention:

```
typedef struct {
    int status;
    char pad[60]; // Pad structure to match standard 64-byte L1 cache line size
} cache_aligned_port_t;
```

User prompt: Help me write a search algorithm for a centralized base station process receiving a congestion report from Node (r, c). If immediate neighbors reported full within time window W, expand the search ring outward hop-by-hop ($2, 3, \dots, N$ hops) to find the nearest available charging node.

Response:

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <stdbool.h>
#include <mpi.h>

// Base Station representation of each grid node's state
typedef struct {
    int available_ports;
    double last_update_time; // Timestamp of the last received MPI message
} NodeState;

// Search result structure returned to caller
typedef struct {
    int row;
    int col;
    int cart_rank;
    int distance;
    bool found;
} SearchResult;

/**
 * Validates whether a node is within grid bounds, updated within time window W,
 * and has available charging capacity.
 */
static inline bool is_node_available(
    int r, int c,
    int total_rows, int total_cols,
    NodeState grid[total_rows][total_cols],
    double current_time,
    double window_W
) {
    // 1. Boundary check
    if (r < 0 || r >= total_rows || c < 0 || c >= total_cols) {
        return false;
    }

    // 2. Time freshness check (must be updated within window W)
    if ((current_time - grid[r][c].last_update_time) > window_W) {
        return false; // Outdated status report; treat as unavailable
    }

    // 3. Capacity check
    return grid[r][c].available_ports > 0;
}

/**
 * Expand search outward hop-by-hop (1 to max_hops) centered at node (origin_r, origin_c).
 * Returns the coordinates and Cartesian rank of the nearest available charging node.
 */
SearchResult search_nearest_charging_node(
    int origin_r, int origin_c,
    int total_rows, int total_cols,
    NodeState grid[total_rows][total_cols],
    double current_time,
    double window_W,
    int max_hops,
    MPI_Comm cart_comm
) {
    SearchResult result = {.row = -1, .col = -1, .cart_rank = MPI_PROC_NULL, .distance = -1, .found = fals
    int best_capacity = -1;
```

```
    // Expand search ring hop-by-hop (Manhattan distance k)
    for (int k = 1; k <= max_hops; k++) {

        // Perimeter traversal for Manhattan distance k: |dr| + |dc| = k
        for (int dr = -k; dr <= k; dr++) {
            int dc1 = k - abs(dr);
            int dc_candidates[2] = {dc1, -dc1};
            int num_dc = (dc1 == 0) ? 1 : 2; // Avoid checking dc=0 twice

            for (int i = 0; i < num_dc; i++) {
                int nr = origin_r + dr;
                int nc = origin_c + dc_candidates[i];

                if (is_node_available(nr, nc, total_rows, total_cols, grid, current_time, window_W)) {
                    // Tie-breaker: Select node at hop k with highest available capacity
                    if (grid[nr][nc].available_ports > best_capacity) {
                        best_capacity = grid[nr][nc].available_ports;
                        result.row = nr;
                        result.col = nc;
                        result.distance = k;
                        result.found = true;
                    }
                }
            }
        }

        // Return immediately if any valid node was found in the current ring k
        if (result.found) {
            int coords[2] = {result.row, result.col};
            MPI_Cart_rank(cart_comm, coords, &result.cart_rank);
            return result;
        }
    }

    // No available node found within max_hops limit
    return result;
}
```

Algorithm Design Details

1. **Perimeter-Only Ring Traversal**: Instead of allocating a queue for full 2D Breadth-First Search (BFS), the algorithm uses exact Manhattan ring traversal at hop distance k:

   $$\text{Ring}(k) = \{(r + \Delta r, c + \Delta c) \mid |\Delta r| + |\Delta c| = k\}$$

   This checks exactly $4k$ grid positions per ring, reducing spatial overhead to $\mathcal{O}(1)$ memory.

2. **Time Window Filtering**: Nodes are evaluated against $T_{\text{now}} - T_{\text{last_update}} \leq W$. If a node fails to report within window W, it is assumed disconnected or stagnant and bypassed during search expansion.

3. **MPI Rank Resolution**: Once an available node is found at ring k, `MPI_Cart_rank` converts the selected 2D grid coordinates $(r_{\text{target}}, c_{\text{target}})$ directly into an MPI Cartesian rank, allowing the base station to send a redirect command immediately via `MPI_Send`.

User prompt: Provide C helper functions using standard time or MPI_Wtime to measure one-way message delays and round-trip times (RTT) across nodes, accounting for potential clock drift between processes.

Response: To measure network delays across separate physical nodes accurately, clock drift must be accounted for:

- **Round-Trip Time (RTT)**: Measured entirely on a single process's local clock ($T_4 - T_1$), making it **natively immune to clock drift**.
- **One-Way Delay**: Direct subtraction of timestamps ($T_{\text{recv}} - T_{\text{send}}$) is biased by unsynchronized node clocks. To resolve this, **Cristian's Algorithm** is used to calibrate the clock offset $\theta = T_{\text{remote}} - T_{\text{local}}$ using a symmetric Ping-Pong exchange before measuring individual message delays.

Implementation Code

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

typedef struct {
    double rtt;            // Round-trip time (seconds)
    double clock_offset;   // Clock offset: (Remote_Clock - Local_Clock)
    double one_way_delay;  // Corrected one-way delay (seconds)
} NetworkMetrics;

// Message header timestamp payload used during offset synchronization
typedef struct {
```

```
    double t1; // Time sent by initiator (Node A clock)
    double t2; // Time received by responder (Node B clock)
    double t3; // Time replied by responder (Node B clock)
} SyncPayload;

/**
 * Calibrates clock offset and measures baseline RTT using Cristian's Algorithm.
 * Must be called cooperatively by node_a (initiator) and node_b (responder).
 */
NetworkMetrics calibrate_clock_and_rtt(int peer_rank, int is_initiator, MPI_Comm comm) {
    NetworkMetrics metrics = {0.0, 0.0, 0.0};

    if (is_initiator) {
        SyncPayload payload;
        double t4;

        // 1. Record T1 on local clock and send to Peer
        payload.t1 = MPI_Wtime();
        MPI_Send(&payload, sizeof(SyncPayload), MPI_BYTE, peer_rank, 100, comm);

        // 4. Receive reply containing T2 and T3 from Peer, record T4 on local clock
        MPI_Recv(&payload, sizeof(SyncPayload), MPI_BYTE, peer_rank, 101, comm, MPI_STATUS_IGNORE);
        t4 = MPI_Wtime();

        // Calculate RTT excluding remote processing time (T3 - T2)
        metrics.rtt = (t4 - payload.t1) - (payload.t3 - payload.t2);

        // Estimate clock offset theta = (Clock_B - Clock_A)
        // theta = [ (T2 - T1) + (T3 - T4) ] / 2
        metrics.clock_offset = ((payload.t2 - payload.t1) + (payload.t3 - t4)) / 2.0;

        // One-way latency under symmetric network path assumption
        metrics.one_way_delay = metrics.rtt / 2.0;

    } else {
        SyncPayload payload;

        // 2. Receive from initiator and immediately record T2 on local clock
        MPI_Recv(&payload, sizeof(SyncPayload), MPI_BYTE, peer_rank, 100, comm, MPI_STATUS_IGNORE);
        payload.t2 = MPI_Wtime();

        // 3. Record T3 on local clock and return payload to initiator
        payload.t3 = MPI_Wtime();
        MPI_Send(&payload, sizeof(SyncPayload), MPI_BYTE, peer_rank, 101, comm);
    }

    return metrics;
}

/**
 * Calculates corrected one-way delay for any arbitrary message payload
 * using a previously calibrated clock offset.
 */
double compute_corrected_one_way_delay(double send_timestamp, double recv_timestamp, double clock_offset)
    // Uncorrected delay biased by clock drift
    double raw_delay = recv_timestamp - send_timestamp;

    // Corrected delay = (T_recv - T_send) - (Clock_Recv - Clock_Send)
    double corrected_delay = raw_delay - clock_offset;

    // Guard against negative delays caused by transient network jitter
    return (corrected_delay < 0.0) ? 0.0 : corrected_delay;
}
```

Usage Example

```
int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0) {
        // Node 0 acts as initiator against Node 1
        NetworkMetrics metrics = calibrate_clock_and_rtt(1, 1, MPI_COMM_WORLD);

        printf("[Node 0] Measured RTT : %.6f ms\n", metrics.rtt * 1000.0);
        printf("[Node 0] Clock Offset : %.6f ms\n", metrics.clock_offset * 1000.0);
        printf("[Node 0] Est. One-Way : %.6f ms\n", metrics.one_way_delay * 1000.0);

        // Standard One-Way Message Test with Embedded Timestamp
        double t_send = MPI_Wtime();
        MPI_Send(&t_send, 1, MPI_DOUBLE, 1, 200, MPI_COMM_WORLD);
```

```
    } else if (rank == 1) {
        // Node 1 acts as responder
        NetworkMetrics metrics = calibrate_clock_and_rtt(0, 0, MPI_COMM_WORLD);

        double t_send;
        MPI_Recv(&t_send, 1, MPI_DOUBLE, 0, 200, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        double t_recv = MPI_Wtime();

        // Node 1 uses negative offset (-clock_offset) because offset is defined as (Clock_1 - Clock_0)
        double delay = compute_corrected_one_way_delay(t_send, t_recv, -metrics.clock_offset);
        printf("[Node 1] Corrected One-Way Delay: %.6f ms\n", delay * 1000.0);
    }

    MPI_Finalize();
    return 0;
}
```

Key Formulas Used

1. **Clock Offset Estimation (θ)**:

$$\theta = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}$$

   Where T_1, T_4 are timestamps on Node A's clock and T_2, T_3 are timestamps on Node B's clock.

2. **One-Way Delay Calibration**:

$$\text{Delay}_{\text{corrected}} = (T_{\text{recv}} - T_{\text{send}}) - \theta_{B-A}$$

User prompt: Help me draft a discussion section comparing grid vs. line topologies under heavy load. Include analysis on why line topologies experience higher report frequencies to the base station due to limited neighbor pathways.

Response: Discussion

1. Architectural Impact of Topological Degree on Local Load Absorption

The fundamental driver of performance divergence between 2D grid and 1D line topologies under heavy load lies in their topological node degrees (d). In a 1D line topology, internal processes possess a degree of $d = 2$ ($\text{Left}, \text{Right}$), whereas a 2D mesh provides $d = 4$ ($\text{Up}, \text{Down}, \text{Left}, \text{Right}$).

Under low-to-moderate charging demand, both topologies handle traffic efficiently via direct neighbor offloading. However, under heavy load, the limited degree of the 1D line creates severe topological constrictions. When a node experiences a sudden spike in EV arrivals, it can only attempt local load balancing along a single spatial axis. If its two direct neighbors are already operating near capacity, local spillover fails immediately.

Conversely, the $d = 4$ connectivity of a 2D mesh provides two additional orthogonal degrees of freedom. This structural redundancy enables localized "load diffusion," where localized congestion dissipates multi-directionally across neighboring nodes before propagating into regional saturation.

2. Escalation Dynamics and Base Station Reporting Frequencies

The structural limitations of the 1D line topology directly translate into a significant increase in fallback reporting frequency to the centralized base station.

```
1D Line Topology (d = 2): Local Saturation Threshold = 2 Occupied Neighbors
   [Node i-1] <---> [ Node i ] <---> [Node i+1]
   (Both saturated -> Immediate Base Station Escalation)

2D Mesh Topology (d = 4): Local Saturation Threshold = 4 Occupied Neighbors
                          [Node i, j-1]
                                ^
                                |
   [Node i-1, j] <---> [ Node i, j ] <---> [Node i+1, j]
                                |
                                v
                          [Node i, j+1]
   (Requires 4 saturated neighbors before Base Station Escalation)
```

Because a 1D node exhausts its immediate local alternatives after just two saturated neighbor checks, the probability of reaching local saturation scales aggressively with traffic density:

- **1D Fast-Path Failure:** In a 1D line, if the probability of an individual node being fully occupied is p, the probability that a congested node's entire immediate neighborhood is full—forcing an explicit central search request—is $P_{\text{report, 1D}} = p^2$.
- **2D Fast-Path Failure:** In a 2D mesh, the same node must encounter four saturated orthogonal neighbors before declaring local exhaustion, yielding an escalation probability of $P_{\text{report, 2D}} = p^4$.

Under heavy load conditions (e.g., $p = 0.8$), a 1D node escalates to the base station in 64% of congestion events (0.8^2), compared to only 40.96% (0.8^4) in a 2D grid. This structural difference causes a high volume of fallback messages to flood the base station in 1D layouts, converting distributed local traffic management into a centralized control-plane bottleneck.

3. Search Ring Expansion and Spatial Search Efficiency

When local neighbors are occupied and a search query reaches the base station, the spatial search efficiency dictates how quickly an alternative node is located.

```
Search Space Growth by Hop Distance (h):

Hop Distance (h)   | 1D Line Perimeter | 2D Mesh Perimeter | 2D Cumulative Nodes
-------------------|-------------------|-------------------|--------------------
1                  | 2 nodes           | 4 nodes           | 4 nodes
2                  | 2 nodes           | 8 nodes           | 12 nodes
3                  | 2 nodes           | 12 nodes          | 24 nodes
h                  | 2 nodes           | 4h nodes          | 2h(h + 1) nodes
```

The expansion dynamics illustrate a major scaling disparity:

- **1D Search Growth:** Expanding the search radius to h hops in a 1D topology evaluates only 2 additional candidate nodes at distance h. The cumulative search space grows strictly linearly:

$$N_{1D}(h) = 2h$$

- **2D Search Growth:** Expanding the Manhattan search ring in a 2D grid evaluates $4h$ candidate nodes at perimeter step h. The cumulative search space grows quadratically:

$$N_{2D}(h) = 2h(h+1)$$

Under heavy system-wide load, finding an available charging port in a 1D line requires searching significantly deeper along the line ($h \propto N$), leading to elevated message routing latencies and larger search ring sizes. In contrast, the 2D mesh expands radially ($h \propto \quad N$), enabling the base station to locate available capacity within a much smaller topological radius.

4. Implications for MPI Communication Overhead

The combination of higher report frequencies and deeper search rings in 1D line topologies introduces distinct MPI overhead patterns:

1. **MPI Point-to-Point Serialization:** The high volume of report messages targeting Rank 0 (Base Station) in 1D runs increases thread contentions and queue depths at the base station process, increasing message handling latency.
2. **Network Diameter and Latency:** The maximum network diameter in a $1 \times N$ line is $N - 1$, compared to $2(\quad N - 1)$ for a balanced $N \times \quad N$ mesh. Inter-node redirection commands issued by the base station suffer from higher end-to-end traversal delays over the 1D network.
3. **Cascading Saturation:** Because 1D nodes lack orthogonal rerouting paths, regional surges trigger contiguous "waves" of saturated nodes. This forces the base station to repeatedly execute long-distance search expansions, generating higher network traffic compared to the localized containment observed in 2D topologies.
